2.3 SQL Constraints

SQL constraints are essential elements in **relational database design** that ensure the **integrity**, **accuracy**, and **reliability** of the data stored in a database. By enforcing specific rules on table columns, SQL constraints help maintain data consistency, preventing invalid data entries and optimizing query performance.

What Are SQL Constraints?

SQL constraints are rules applied to **columns** or **tables** in a **relational database** to limit the type of data that can be **inserted**, **updated**, or **deleted**. These rules ensure the data is valid, consistent, and adheres to the business logic or **database requirements**. Constraints can be enforced during table creation or later using the **ALTER TABLE** statement. They play a vital role in maintaining the quality and integrity of your database.

Types of SQL Constraints

SQL provides several types of constraints to manage different aspects of data integrity. These constraints are essential for ensuring that data meets the requirements of **accuracy**, **consistency**, and **validity**. Let's go through each of them with detailed explanations and examples.

1. NOT NULL Constraint

The **NOT NULL** constraint ensures that a column cannot contain NULL values. This is particularly important for columns where a value is essential for identifying records or performing calculations. If a column is defined as **NOT NULL**, every row must include a value for that column.

Example:

```
CREATE TABLE Student
(
    ID int(6) NOT NULL,
    NAME varchar(10) NOT NULL,
    ADDRESS varchar(20)
);
```

Explanation: In the above example, both the ID and NAME columns are defined with the **NOT NULL** constraint, meaning every student must have an ID and NAME value.

2. UNIQUE Constraint

The **UNIQUE** constraint ensures that all values in a column are distinct across all rows in a table. Unlike the **PRIMARY KEY**, which requires uniqueness and does not allow NULLs, the UNIQUE constraint allows NULL values but still enforces uniqueness for non-NULL entries.

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL UNIQUE,
NAME varchar(10),
ADDRESS varchar(20)
);
```

Explanation: Here, the ID column must have unique values, ensuring that no two students can share the same ID. We can have more than one **UNIQUE** constraint in a table.

3. PRIMARY KEY Constraint

A **PRIMARY KEY** constraint is a combination of the **NOTNULL** and **UNIQUE** constraints. It uniquely identifies each row in a table. A table can only have one PRIMARY KEY, and it cannot accept NULL values. This is typically used for the column that will serve as the identifier of records.

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL UNIQUE,
NAME varchar(10),
ADDRESS varchar(20),
PRIMARY KEY(ID)
);
```

Explanation: In this case, the ID column is set as the primary key, ensuring that each student's ID is unique and cannot be NULL.

4. FOREIGN KEY Constraint

A **FOREIGN KEY** constraint links a column in one table to the **primary key** in another table. This relationship helps maintain **referential integrity** by ensuring that the value in the foreign key column matches a valid record in the referenced table.

Orders Table:

O_ID	ORDER_NO	C_ID
1	2253	3
2	3325	3

O_ID	ORDER_NO	C_ID
3	4521	2
4	8532	1

Customers Table:

C_ID	NAME	ADDRESS
1	RAMESH	DELHI
2	SURESH	NOIDA
3	DHARMESH	GURGAON

As we can see clearly that the field **C_ID** in **Orders table** is the **primary key** in Customers table, i.e. it uniquely identifies each row in the **Customers** table. Therefore, it is a Foreign Key in Orders table.

Example:

```
CREATE TABLE Orders
(
O_ID int NOT NULL,
ORDER_NO int NOT NULL,
C_ID int,
PRIMARY KEY (O_ID),
FOREIGN KEY (C_ID) REFERENCES Customers(C_ID)
)
```

Explanation: In this example, the C_ID column in the Orders table is a foreign key that references the C_ID column in the Customers table. This ensures that only valid customer IDs can be inserted into the Orders table.

5. CHECK Constraint

The **CHECK** constraint allows us to specify a condition that data must satisfy before it is inserted into the table. This can be used to enforce rules, such as ensuring that a column's value meets certain criteria (e.g., age must be greater than 18)

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
AGE int NOT NULL CHECK (AGE >= 18)
);
```

Explanation: In the above table, the **CHECK** constraint ensures that only students aged 18 or older can be inserted into the table.

6. DEFAULT Constraint

The **DEFAULT** constraint provides a default value for a column when no value is specified during insertion. This is useful for ensuring that certain columns always have a meaningful value, even if the user does not provide one

Example:

```
CREATE TABLE Student
(
ID int(6) NOT NULL,
NAME varchar(10) NOT NULL,
AGE int DEFAULT 18
);
```